

Matemática Financeira — Teoria Completa com Python

Capítulo 1 — Fundamentos: O Valor do Dinheiro no Tempo

1.1. O Princípio Fundamental

O **valor do dinheiro no tempo** (time value of money — TVM) é o conceito mais importante de toda a matemática financeira. Ele afirma que **uma unidade monetária hoje vale mais do que a mesma unidade monetária em qualquer data futura**.

Por que isso acontece?

1. **Custo de oportunidade** — Se você tem R\$ 1.000 hoje, pode investi-los e obter um retorno. Se só receber esse valor daqui a um ano, perdeu a oportunidade de ganhar juros nesse período.
2. **Inflação** — A tendência geral de aumento de preços faz com que o poder de compra da moeda diminua ao longo do tempo. Com R\$ 1.000 hoje você compra mais do que compraria com R\$ 1.000 daqui a um ano.
3. **Risco** — Existe incerteza quanto ao recebimento futuro. O devedor pode não pagar, a empresa pode quebrar, o governo pode mudar as regras. Essa incerteza precisa ser remunerada.

Desses três fatores, o mais relevante para a matemática financeira é o **custo de oportunidade**, pois ela independe da inflação ou do risco específico da operação.

1.2. Conceitos Primitivos

A matemática financeira opera com alguns conceitos fundamentais que devem ser absolutamente dominados antes de qualquer avanço.

Conceito	Símbolo	Nomes Alternativos	Definição
Capital	C, PV	Valor Presente, Principal, Valor Atual	Quantia disponível ou devida na data de referência inicial da operação
Montante	M, FV	Valor Futuro, Valor de Resgate, Valor Nominal	Quantia disponível ou devida na data de referência final da operação
Juros	J	Remuneração, Encargos, Rendimento	Diferença entre o montante e o capital: $J = FV - PV$
Taxa de Juros	i	Taxa de juros, Rentabilidade, Custo	Razão entre os juros de um período e o capital que os produziu
Período	n	Prazo, Tempo, Duração	Intervalo de tempo ao qual a taxa de juros se refere

A **taxa de juros** merece atenção especial. Ela pode ser expressa de duas formas equivalentes:

- **Forma percentual:** $i = 12\% \text{ a.a.}$ (lê-se “12 por cento ao ano”)
- **Forma decimal (unitária):** $i = 0,12$ (usada em fórmulas)

A conversão é simples:

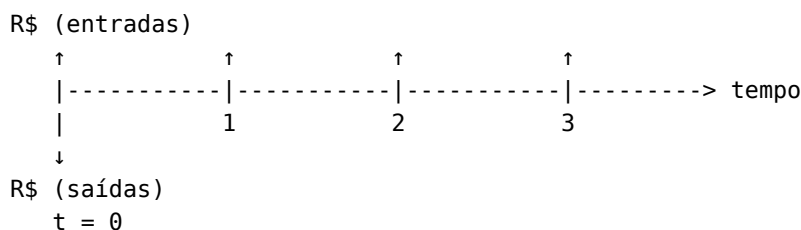
$$i_{decimal} = \frac{i_{percentual}}{100}$$

Importante: a taxa de juros e o período devem estar sempre na **mesma unidade de tempo**. Se a taxa é mensal, o período deve ser medido em meses. Se é anual, em anos.

1.3. Fluxo de Caixa

O **fluxo de caixa** (FC, cash flow — CF) é a representação das entradas e saídas de dinheiro ao longo do tempo. É a ferramenta gráfica e analítica mais importante da matemática financeira.

Convenção: - **Entradas de caixa** (recebimentos) → seta para **cima (+)** - **Saídas de caixa** (pagamentos) → seta para **baixo (-)** - **Linha horizontal** → escala do tempo - **t = 0** → momento presente



Tipos de fluxo de caixa:

Tipo	Descrição	Exemplo
Uniforme	Prestações iguais e periódicas	Financiamento em 12 parcelas fixas
Não uniforme	Valores diferentes em cada período	Fluxo de caixa de um projeto
Postecipado	Pagamentos ao final do período	Prestação sem entrada
Antecipado	Pagamentos no início do período	Aluguel pago no começo do mês
Diferido	Pagamentos com carência	Primeira prestação após 3 meses
Perpétuo	Pagamentos infinitos	Ação que paga dividendos para sempre

1.4. Regimes de Capitalização

O regime de capitalização define **como os juros são calculados e incorporados ao capital** ao longo do tempo. Existem dois regimes fundamentais:

Regime de Capitalização Simples (Juros Simples): - Os juros de cada período incidem **apenas sobre o capital inicial** - O crescimento é **linear** (progressão aritmética) - Usado principalmente em operações de curto prazo (descontos, hot money)

Regime de Capitalização Composta (Juros Compostos): - Os juros de cada período incidem **sobre o montante do período anterior** - Ocorre “juros sobre juros” (anatocismo) - O crescimento é **exponencial** (progressão geométrica) - É o regime **universalmente usado** no sistema financeiro

```
import numpy as np
import matplotlib.pyplot as plt

def regime_comparativo(pv=1000, i=0.02, n_max=30):
    """
    Compara os dois regimes de capitalização graficamente.
```

```

pv: capital inicial
i: taxa de juros por período
n_max: número máximo de períodos
"""

periodos = range(n_max + 1)
simples = [pv * (1 + i * n) for n in periodos]
compostos = [pv * (1 + i) ** n for n in periodos]

plt.figure(figsize=(12, 5))
plt.plot(periodos, simples, 'b--', label='Juros Simples (linear)', linewidth=2)
plt.plot(periodos, compostos, 'r-', label='Juros Compostos (exponencial)',
linewidth=2)
plt.fill_between(periodos, simples, compostos, alpha=0.1, color='purple',
label='Diferença (ganho do composto)')
plt.axhline(pv, color='gray', linestyle=':', alpha=0.5)
plt.xlabel('Períodos (n)')
plt.ylabel('Montante (R$)')
plt.title(f'Capitalização Simples vs Composta – PV = R${pv:,.0f}, i =
{i*100:.0f}%')
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

Observação fundamental: Para $n = 1$, os dois regimes produzem o **mesmo montante**. Para $n > 1$, os juros compostos produzem montante **maior** (e a diferença cresce com n). Para $n < 1$ (frações de período), os juros simples podem produzir montante maior que o composto, dependendo da convenção adotada.

Capítulo 2 — Juros Simples

2.1. Dedução das Fórmulas

Seja PV o capital inicial aplicado a uma taxa i por período, durante n períodos, no regime de juros simples.

Período 1: $J_1 = PV \times i$

Período 2: $J_2 = PV \times i$ (ainda sobre o capital inicial)

Período n : $J_n = PV \times i$

O juro total é simplesmente a soma dos juros de cada período:

$$J = J_1 + J_2 + \dots + J_n = PV \times i \times n$$

O montante (valor futuro) é:

$$FV = PV + J = PV + PV \times i \times n = PV \times (1 + i \times n)$$

Fórmulas fundamentais (juros simples):

Objetivo	Fórmula
Juros totais	$J = PV \times i \times n$
Montante	$FV = PV \times (1 + i \times n)$
Capital inicial	$PV = \frac{FV}{1+i \times n}$
Taxa de juros	$i = \frac{FV/PV - 1}{n}$
Período	$n = \frac{FV/PV - 1}{i}$

```
def juros_simples(pv=None, fv=None, i=None, n=None):
    """
    Calcula qualquer termo faltante nos juros simples.
    Forneça exatamente 3 dos 4 parâmetros.
    """
    if pv is None:
        return fv / (1 + i * n)
    elif fv is None:
        return pv * (1 + i * n)
    elif i is None:
        return (fv / pv - 1) / n
    elif n is None:
        return (fv / pv - 1) / i

# Exemplo: Montei R$1.100 a partir de R$1.000 em 5 meses. Qual a taxa?
i = juros_simples(pv=1000, fv=1100, n=5)
print(f"Taxa: {i*100:.2f}% ao mês")

Taxa: 2.00% ao mês
```

2.2. Taxas Proporcionais (Juros Simples)

Duas taxas são ditas **proporcionais** quando a relação entre elas é igual à relação entre os períodos a que se referem. Em juros simples, taxas proporcionais são **equivalentes** (produzem o mesmo montante para o mesmo capital e mesmo prazo total).

$$\frac{i_1}{i_2} = \frac{n_1}{n_2}$$

Convenções de ano: - **Ano comercial (bancário):** 360 dias, 12 meses de 30 dias - **Ano civil (exato):** 365 dias (ou 366 em ano bissexto)

```
def taxas_proporcionais():
    """
    Tabela de taxas proporcionais para 24% a.a.
    """
    i_aa = 0.24
    print(f"Taxa de referência: {i_aa*100}% a.a.\n")
    print(f"{'Período':<15} {'Taxa Proporcional':<20} {'Cálculo'}")
    print("-" * 55)
    print(f"{'Ano':<15} {i_aa*100:<19.2f}% {i_aa}")
    print(f"{'Semestre':<15} {i_aa/2*100:<19.2f}% {i_aa}/2")
    print(f"{'Trimestre':<15} {i_aa/4*100:<19.2f}% {i_aa}/4")
    print(f"{'Mês':<15} {i_aa/12*100:<19.2f}% {i_aa}/12")
    print(f"{'Dia (comercial)':<15} {i_aa/360*100:<19.4f}% {i_aa}/360")
    print(f"{'Dia (exato)':<15} {i_aa/365*100:<19.4f}% {i_aa}/365")

taxas_proporcionais()
```

Taxa de referência: 24.0% a.a.

Período	Taxa Proporcional	Cálculo
-----	-----	-----
Ano	24.00%	0.24
Semestre	12.00%	0.24/2
Trimestre	6.00%	0.24/4
Mês	2.00%	0.24/12
Dia (comercial)	0.0667%	0.24/360
Dia (exato)	0.0658%	0.24/365

2.3. Juros Exatos vs Juros Comerciais

Juros comerciais (ordinários): utilizam o ano de 360 dias. São os mais usados em operações bancárias de curto prazo.

$$J_{comercial} = PV \times i_{anual} \times \frac{dias}{360}$$

Juros exatos: utilizam o ano civil de 365 dias.

$$J_{exato} = PV \times i_{anual} \times \frac{dias}{365}$$

```
def comparar_juros(pv, i_aa, dias):
    j_com = pv * i_aa * dias / 360
    j_exa = pv * i_aa * dias / 365
    print(f"Capital: R${pv:,.2f} | Taxa: {i_aa*100:.1f}% a.a. | Período: {dias} dias\n")
    print(f"Juros comerciais (360 dias): R${j_com:,.2f}")
    print(f"Juros exatos (365 dias): R${j_exa:,.2f}")
    print(f"Diferença: R${j_com - j_exa:,.4f}")
    print(f"Relação: {j_com/j_exa:.4f} (={360/365:.4f})")
```

```
comparar_juros(10000, 0.12, 45)
```

Capital: R\$10,000.00 | Taxa: 12.0% a.a. | Período: 45 dias

Juros comerciais (360 dias): R\$150.00
Juros exatos (365 dias): R\$147.95
Diferença: R\$2.0548
Relação: 1.0139 (=0.9863)

Observação: Os juros comerciais são sempre **maiores** que os exatos para um mesmo capital, taxa e prazo, pois o ano de 360 dias divide o numerador por um denominador menor. A relação entre eles é sempre $J_{com}/J_{ex} = 365/360 \approx 1,0139$.

2.4. Descontos Simples

Desconto é o abatimento concedido pelo pagamento antecipado de um título. Existem duas modalidades fundamentais.

2.4.1. Desconto Comercial (por Fora)

O desconto é calculado **sobre o valor nominal** (valor futuro, de face) do título. É o mais usado pelo mercado bancário brasileiro.

$$D_c = FV \times i \times n$$

$$PV = FV - D_c = FV \times (1 - i \times n)$$

Onde: - FV = valor nominal (valor de face, valor no vencimento) - PV = valor atual (valor descontado, valor líquido) - i = taxa de desconto - n = período de antecipação

Restrição: a operação só faz sentido se $i \times n < 1$. Caso contrário, PV seria zero ou negativo.

2.4.2. Desconto Racional (por Dentro)

O desconto é calculado **sobre o valor presente**. É o “verdadeiro” desconto, pois usa a mesma lógica dos juros.

$$D_r = PV \times i \times n$$

$$PV = \frac{FV}{1 + i \times n}$$

2.4.3. Relação entre os Descontos

- $D_c > D_r$ para o mesmo FV , i e n
- $PV_c < PV_r$ (o desconto comercial concede um valor líquido menor)

```
def descontos(fv, i, n):
    """
    Calcula os dois tipos de desconto simples.
    """
    d_com = fv * i * n
    pv_com = fv - d_com
    pv_rac = fv / (1 + i * n)
    d_rac = fv - pv_rac

    print(f"{'Variável':<20} {'Comercial':<15} {'Racional':<15}")
    print("-" * 50)
    print(f"{'Valor Nominal (FV)':<20} R\${fv:<10.2f} R\${fv:<10.2f}")
    print(f"{'Valor Atual (PV)':<20} R\${pv_com:<10.2f} R\${pv_rac:<10.2f}")
    print(f"{'Desconto (D)':<20} R\${d_com:<10.2f} R\${d_rac:<10.2f}")
    print(f"{'Taxa efetiva':<20} {(d_com/pv_com)*100:<13.2f}% {(d_rac/
pv_rac)*100:<13.2f}%")
```

```
descontos(5000, 0.03, 2)
```

Variável	Comercial	Racional

Valor Nominal (FV)	R\$5000.00	R\$5000.00
Valor Atual (PV)	R\$4700.00	R\$4716.98
Desconto (D)	R\$300.00	R\$283.02
Taxa efetiva	6.38%	6.00%

2.5. Equivalência entre Taxa de Desconto e Taxa de Juros

A taxa de desconto d e a taxa de juros i são diferentes. Para um mesmo valor nominal FV e mesmo prazo n :

$$i = \frac{d}{1 - d \times n}$$

$$d = \frac{i}{1 + i \times n}$$

```
def taxa_desconto_para_juros(d, n):
    """Converte taxa de desconto comercial para taxa de juros efetiva"""
    return d / (1 - d * n)
```

```
def taxa_juros_para_desconto(i, n):
    """Converte taxa de juros para taxa de desconto comercial"""
    return i / (1 + i * n)

# Uma taxa de desconto de 3% a.m. em 2 meses equivale a que taxa de juros?
d, n = 0.03, 2
i_eq = taxa_desconto_para_juros(d, n)
print(f"Taxa de desconto {d*100}% a.m. = Taxa de juros {i_eq*100:.2f}% a.m.")
Taxa de desconto 3.0% a.m. = Taxa de juros 3.19% a.m.
```

2.6. Prazo Médio de um Conjunto de Títulos

Quando se deseja substituir vários títulos por um único, calcula-se o **prazo médio**, que é a média ponderada dos prazos pelos valores nominais.

$$n_{\text{médio}} = \frac{\sum(FV_k \times n_k)}{\sum FV_k}$$

```
def prazo_medio(titulos):
    """
    titulos: lista de (valor_nominal, prazo_em_meses)
    """
    soma_valores = sum(fv for fv, _ in titulos)
    soma_ponderada = sum(fv * n for fv, n in titulos)
    return soma_ponderada / soma_valores

titulos = [(5000, 30), (8000, 60), (7000, 90)]
pm = prazo_medio(titulos)
print(f"Prazo médio: {pm:.2f} dias")
Prazo médio: 61.50 dias
```

Capítulo 3 – Juros Compostos

3.1. Dedução das Fórmulas

No regime de juros compostos, a **base de cálculo dos juros se renova a cada período**. Os juros do período anterior são incorporados ao capital para o cálculo dos juros do período seguinte.

Período 1:

$$FV_1 = PV + PV \times i = PV \times (1 + i)$$

Período 2:

$$FV_2 = FV_1 + FV_1 \times i = FV_1 \times (1 + i) = PV \times (1 + i)^2$$

Período n:

$$FV_n = PV \times (1 + i)^n$$

Fórmulas fundamentais (juros compostos):

Objetivo	Fórmula
Montante	$FV = PV \times (1 + i)^n$
Capital inicial	$PV = \frac{FV}{(1+i)^n} = FV \times (1 + i)^{-n}$
Juros totais	$J = FV - PV = PV \times [(1 + i)^n - 1]$
Taxa de juros	$i = \left(\frac{FV}{PV}\right)^{1/n} - 1$
Período	$n = \frac{\ln(FV/PV)}{\ln(1+i)}$

```
import math
```

```
def juros_compostos(pv=None, fv=None, i=None, n=None):
    """
    Calcula qualquer termo faltante nos juros compostos.
    Forneça exatamente 3 dos 4 parâmetros.
    """
    if pv is None:
        return fv / (1 + i) ** n
    elif fv is None:
        return pv * (1 + i) ** n
    elif i is None:
        return (fv / pv) ** (1 / n) - 1
    elif n is None:
        return math.log(fv / pv) / math.log(1 + i)
```

```
# Exemplo 1: R$5.000 a 2% a.m. por 12 meses
fv = juros_compostos(pv=5000, i=0.02, n=12)
print(f"Ex.1: R$5.000 a 2% a.m. por 12m = R${fv:.2f}")
```

```
# Exemplo 2: Qual taxa transforma R$1.000 em R$2.000 em 24 meses?
i = juros_compostos(pv=1000, fv=2000, n=24)
print(f"Ex.2: Taxa para dobrar em 24m = {i*100:.4f}% a.m.")
```

```
# Exemplo 3: Quanto tempo para R$1.000 virar R$3.000 a 1,5% a.m.?
n = juros_compostos(pv=1000, fv=3000, i=0.015)
print(f"Ex.3: Tempo para triplicar a 1,5% a.m. = {n:.2f} meses")
```

Ex.1: R\$5.000 a 2% a.m. por 12m = R\$6341.21

Ex.2: Taxa para dobrar em 24m = 2.93% a.m.

Ex.3: Tempo para triplicar a 1,5% a.m. = 73.79 meses

3.2. A Regra dos 72

Uma aproximação útil para estimar o tempo de duplicação de um capital a uma dada taxa:

$$n_{\text{dobro}} \approx \frac{72}{i_{\text{percentual}}}$$

```
def regra_72(taxa_percentual):
    n_aproximado = 72 / taxa_percentual
    n_exato = math.log(2) / math.log(1 + taxa_percentual / 100)
    erro = abs(n_aproximado - n_exato) / n_exato * 100
    return n_aproximado, n_exato, erro

for taxa in [1, 2, 3, 5, 10, 15, 20]:
    aprox, exato, erro = regra_72(taxa)
```



```
print(f"Taxa {taxa:>2}%: Regra 72={aprox:.2f}m, Exato={exato:.2f}m, Erro={erro:.1f}%")
```

```
Taxa 1%: Regra 72=72.00m, Exato=69.66m, Erro=3.4%
Taxa 2%: Regra 72=36.00m, Exato=35.00m, Erro=2.9%
Taxa 3%: Regra 72=24.00m, Exato=23.45m, Erro=2.4%
Taxa 5%: Regra 72=14.40m, Exato=14.21m, Erro=1.4%
Taxa 10%: Regra 72=7.20m, Exato=7.27m, Erro=1.0%
Taxa 15%: Regra 72=4.80m, Exato=4.96m, Erro=3.2%
Taxa 20%: Regra 72=3.60m, Exato=3.80m, Erro=5.3%
```

3.3. Taxas Equivalentes (Juros Compostos)

Em juros compostos, **taxas proporcionais NÃO são equivalentes**. Duas taxas são equivalentes quando, aplicadas ao mesmo capital pelo mesmo prazo total, produzem o mesmo montante, **mas em períodos de capitalização diferentes**.

A relação fundamental:

$$(1 + i_1)^{n_1} = (1 + i_2)^{n_2}$$

Generalizando:

$$i_{maior} = (1 + i_{menor})^k - 1$$

$$i_{menor} = (1 + i_{maior})^{1/k} - 1$$

Onde k é o número de períodos menores contidos no período maior.

```
def taxa_equivalente(i, periodo_origem, periodo_destino):
    """
    Converte taxas no regime composto.
    Períodos: 'ano', 'semestre', 'trimestre', 'mes', 'dia'
    """
    periodos = {'ano': 1, 'semestre': 2, 'trimestre': 4, 'mes': 12, 'dia': 360}
    k = periodos[periodo_destino] / periodos[periodo_origem]
    return (1 + i) ** (1 / k) - 1 if k > 0 else (1 + i) ** (-k) - 1

# 24% a.a. para diversas bases
i_aa = 0.24
print("Conversão de 24% a.a. para outras bases (regime composto):")
print(f" Mensal: {taxa_equivalente(i_aa, 'ano', 'mes')*100:.4f}% a.m.")
print(f" Diária: {taxa_equivalente(i_aa, 'ano', 'dia')*100:.4f}% a.d.")
print(f" Semestral: {taxa_equivalente(i_aa, 'ano', 'semestre')*100:.4f}% a.s.")

# E no caminho inverso: 2% a.m. para anual
i_am = 0.02
i_aa_equiv = taxa_equivalente(i_am, 'mes', 'ano')
print(f"\n2% a.m. = {i_aa_equiv*100:.2f}% a.a. (equivalente composto)")
print(f"2% a.m. = {i_am*12*100:.2f}% a.a. (proporcional simples – incorreto em composto)")

Conversão de 24% a.a. para outras bases (regime composto):
Mensal: 1.8088% a.m.
Diária: 0.0598% a.d.
Semestral: 11.36% a.s.

2% a.m. = 26.82% a.a. (equivalente composto)
2% a.m. = 24.00% a.a. (proporcional simples – incorreto em composto)
```

3.4. Taxa Nominal vs Taxa Efetiva

Taxa Nominal: é a taxa declarada no contrato, mas cujo período de capitalização é **diferente** do período a que ela se refere. Exemplo: “24% ao ano **capitalizados mensalmente**”.

Taxa Efetiva: é a taxa que efetivamente será aplicada no período considerado.

Para converter nominal em efetiva:

$$i_{efetiva} = \left(1 + \frac{i_{nominal}}{k}\right)^k - 1$$

Onde k = número de capitalizações dentro do período nominal.

```
def nominal_para_efetiva(i_nominal, capitalizacoes_por_periodo):
    """
    Converte taxa nominal para taxa efetiva.
    Ex: nominal 24% a.a. capitalizada mensalmente -> capitalizacoes_por_periodo = 12
    """
    return (1 + i_nominal / capitalizacoes_por_periodo) ** capitalizacoes_por_periodo - 1

def efetiva_para_nominal(i_efetiva, capitalizacoes_por_periodo):
    """
    Converte taxa efetiva para taxa nominal.
    """
    return ((1 + i_efetiva) ** (1 / capitalizacoes_por_periodo) - 1) * capitalizacoes_por_periodo

print("Conversões de taxas nominais para efetivas:")
for taxa_nominal in [0.12, 0.18, 0.24, 0.30]:
    for cap in [2, 4, 6, 12]:
        ef = nominal_para_efetiva(taxa_nominal, cap)
        print(f" {taxa_nominal*100:.0f}% a.a. cap. {cap}x/ano -> efetiva {ef*100:.2f}% a.a.")
    print()
```

Implicação importante: quanto **maior** o número de capitalizações dentro do período, **maior** será a taxa efetiva. O limite teórico é a **capitalização contínua**:

$$i_{contínua} = e^{i_{nominal}} - 1$$

onde $e \approx 2,71828$ (número de Euler).

```
import math

def capitalizacao_continua(i_nominal):
    return math.exp(i_nominal) - 1

i_nom = 0.24
print(f"24% a.a. com capitalização contínua: {capitalizacao_continua(i_nom)*100:.4f}% a.a.")
print(f"24% a.a. com capitalização mensal: {nominal_para_efetiva(i_nom, 12)*100:.2f}% a.a.")
print(f"24% a.a. com capitalização diária: {nominal_para_efetiva(i_nom, 360)*100:.4f}% a.a.")
```

3.5. Juros Compostos com Períodos Fracionários

Em operações reais, o prazo n pode não ser inteiro. Existem duas convenções:

Convenção Linear (exponencial para parte inteira, linear para fração):

$$FV = PV \times (1 + i)^{n_{int}} \times (1 + i \times n_{frac})$$

Convenção Exponencial (potência para o prazo total):

$$FV = PV \times (1 + i)^n$$

```
def fv_convencao(pv, i, n, convencao='exponencial'):
    n_int = int(n)
    n_frac = n - n_int
    if convencao == 'linear':
        return pv * (1 + i) ** n_int * (1 + i * n_frac)
    else:
        return pv * (1 + i) ** n

# R$1.000 a 2% a.m. por 6 meses e 15 dias (n = 6.5)
pv, i, n = 1000, 0.02, 6.5
print(f"Convenção exponencial: R${fv_convencao(pv, i, n, 'exponencial'):.2f}")
print(f"Convenção linear: R${fv_convencao(pv, i, n, 'linear'):.2f}")

Convenção exponencial: R$1141.17
Convenção linear: R$1142.39
```

A convenção linear sempre produz montante **levemente maior**, e é a mais usada no mercado financeiro brasileiro.

Capítulo 4 – Séries de Pagamentos

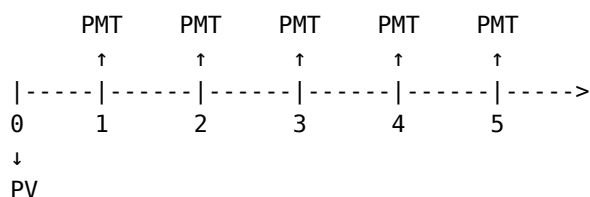
4.1. Classificação das Séries

Uma **série de pagamentos** (ou anuidade) é uma sucessão de capitais disponíveis em diferentes datas. Classifica-se segundo vários critérios:

Critério	Tipos
Período	Periódicos (intervalos iguais) vs Não Periódicos
Valor	Constantes (uniformes) vs Variáveis
Início	Postecipados (1º pgto ao final do 1º período) vs Antecipados (1º pgto no início) vs Diferidos (carência)
Duração	Finitas (temporárias) vs Infinitas (perpétuas)
Forma de cálculo	Imediatas vs Diferidas

4.2. Série Uniforme Postecipada

A mais comum: pagamentos iguais ao **final** de cada período.



Valor Presente:

Cada pagamento PMT é descontado a valor presente:

$$PV = \frac{PMT}{(1+i)^1} + \frac{PMT}{(1+i)^2} + \dots + \frac{PMT}{(1+i)^n}$$

A soma da progressão geométrica resulta em:

$$PV = PMT \times \frac{(1+i)^n - 1}{i \times (1+i)^n}$$

Valor Futuro:

$$FV = PMT \times \frac{(1+i)^n - 1}{i}$$

Prestação (PMT):

$$PMT = PV \times \frac{i \times (1+i)^n}{(1+i)^n - 1}$$

O fator $\frac{i \times (1+i)^n}{(1+i)^n - 1}$ é chamado **Fator de Recuperação de Capital (FRC)**.

```
def serie_postecipada(pv=None, pmt=None, fv=None, i=None, n=None):
    """
    Calcula séries uniformes postecipadas.
    """
    if pmt is None:
        if pv is not None:
            fator = (i * (1 + i) ** n) / ((1 + i) ** n - 1)
            return pv * fator
        else:
            fator = i / ((1 + i) ** n - 1)
            return fv * fator
    elif pv is None and fv is None:
        return None

# Exemplo: Financiar R$30.000 em 24x a 1,8% a.m.
pv, i, n = 30000, 0.018, 24
pmt = serie_postecipada(pv=pv, i=i, n=n)
total_pago = pmt * n
juros_pagos = total_pago - pv

print(f"Valor financiado: R${pv:,.2f}")
print(f"Taxa: {i*100:.1f}% a.m.")
print(f"Prazo: {n} meses")
print(f"Prestação: R${pmt:.2f}")
print(f"Total pago: R${total_pago:,.2f}")
print(f"Juros totais: R${juros_pagos:,.2f}")
print(f"Taxa efetiva total: {(total_pago/pv - 1)*100:.2f}%")
print(f"Custo mensal médio: R${juros_pagos/n:.2f} de juros por mês")

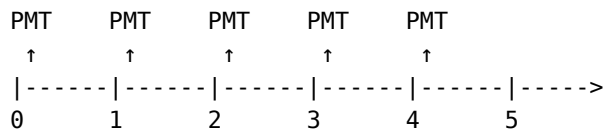
Valor financiado: R$30,000.00
Taxa: 1.8% a.m.
Prazo: 24 meses
Prestação: R$1,549.65
Total pago: R$37,191.60
Juros totais: R$7,191.60
```

Taxa efetiva total: 23.97%

Custo mensal médio: R\$299.65 de juros por mês

4.3. Série Uniforme Antecipada

Pagamentos no **início** de cada período. A primeira parcela é paga no ato (entrada).



Valor Presente:

$$PV_{ant} = PMT \times \frac{(1+i)^n - 1}{i \times (1+i)^n} \times (1+i)$$

Ou alternativamente:

$$PV_{ant} = PMT + PMT \times \frac{(1+i)^{n-1} - 1}{i \times (1+i)^{n-1}}$$

```
def serie_antecipada(pv, i, n):  
    """Calcula prestação de série antecipada (com entrada)."""  
    fator = ((1 + i) ** n - 1) / (i * (1 + i) ** n)  
    return pv / (fator * (1 + i))
```

```
# Mesmo financiamento com entrada (série antecipada)  
pv, i, n = 30000, 0.018, 24  
pmt_ant = serie_antecipada(pv, i, n)  
print(f"Série postecipada (sem entrada): R\${pmt:.2f}")  
print(f"Série antecipada (com entrada): R\${pmt_ant:.2f}")  
print(f"Diferença: R\${pmt - pmt_ant:.2f} por mês")
```

Série postecipada (sem entrada): R\$1,549.65
Série antecipada (com entrada): R\$1,522.25
Diferença: R\$27.40 por mês

4.4. Série Diferida (com Carência)

Quando há um período de carência antes do início dos pagamentos.

$$PV_{diferida} = \frac{PMT \times \frac{(1+i)^n - 1}{i \times (1+i)^n}}{(1+i)^m}$$

Onde m = número de períodos de carência.

```
def serie_diferida(pv, i, n, carencia):  
    """  
    Calcula prestação com período de carência.  
    carencia: número de períodos sem pagamento  
    """  
    fator_padrao = (i * (1 + i) ** n) / ((1 + i) ** n - 1)  
    fator_carencia = (1 + i) ** carencia  
    pmt_na_carencia = pv * fator_padrao * fator_carencia  
    return pmt_na_carencia  
  
# R$30.000, 1,8% a.m., 24 parcelas, carência de 3 meses  
pmt_dif = serie_diferida(30000, 0.018, 24, 3)  
print(f"Prestação com 3 meses de carência: R\${pmt_dif:.2f}")
```

```
print(f"Prestação sem carência: R${pmt:.2f}")
print(f"Diferença: R${pmt_dif - pmt:.2f}")
```

Prestação com 3 meses de carência: R\$1,634.92
Prestação sem carência: R\$1,549.65
Diferença: R\$85.27

4.5. Perpetuidades

Uma perpetuidade é uma série de pagamentos **infinitos** e iguais.

Valor Presente da Perpetuidade:

$$PV = \frac{PMT}{i}$$

```
def perpetuidade(pmt=None, i=None, pv=None):
    if pv is None:
        return pmt / i
    elif pmt is None:
        return pv * i
    elif i is None:
        return pmt / pv

# Quanto preciso ter para receber R$5.000/mês para sempre a 0,8% a.m.?
pv = perpetuidade(pmt=5000, i=0.008)
print(f"Valor necessário para perpetuidade: R${pv:,.2f}")

# Se tenho R$1.000.000, quanto posso sacar por mês a 0,6% a.m.?
pmt = perpetuidade(pv=1000000, i=0.006)
print(f"Saque mensal perpétuo: R${pmt:.2f}")

Valor necessário para perpetuidade: R$625,000.00
Saque mensal perpétuo: R$6,000.00
```

4.6. Perpetuidade com Crescimento (Modelo de Gordon)

Quando os pagamentos crescem a uma taxa constante g :

$$PV = \frac{PMT}{i - g}$$

Restrição: $i > g$ (senão o valor presente explode para infinito).

```
def perpetuidade_crescimento(pmt, i, g):
    if i <= g:
        raise ValueError("i deve ser maior que g")
    return pmt / (i - g)

# Empresa pagará dividendo de R$2,00 este ano, crescendo 4% a.a., Ke=11% a.a.
pv = perpetuidade_crescimento(2, 0.11, 0.04)
print(f"Preço justo da ação (Gordon): R${pv:.2f}")

# Qual a taxa de crescimento implícita se a ação está a R$40 e próximo dividendo é R$3?
# PV = PMT / (i - g) -> g = i - PMT/PV
g = 0.11 - 3/40
print(f"Crescimento implícito no preço de R$40,00: {g*100:.2f}% a.a.")
```

Capítulo 5 — Sistemas de Amortização

5.1. Conceitos Gerais

Amortização é o processo de **devolução do capital emprestado** por meio de pagamentos periódicos. Cada prestação (PMT) é composta de duas partes:

$$Prestação = Amortização + Juros$$

$$PMT_k = A_k + J_k$$

Onde: - A_k = parcela que abate o saldo devedor (amortização do principal) - J_k = parcela que remunera o credor (juros sobre o saldo devedor)

5.2. Sistema Price (Sistema Francês)

Características: - Prestações **constantes** (fixas) durante todo o período - Juros **decrescentes** com o tempo - Amortização **crescente** com o tempo - Amplamente usado em financiamento imobiliário, consórcios, crédito direto

Cálculo da prestação:

$$PMT = PV \times \frac{i \times (1 + i)^n}{(1 + i)^n - 1}$$

Evolução da dívida:

$$J_k = SD_{k-1} \times i$$

$$A_k = PMT - J_k$$

$$SD_k = SD_{k-1} - A_k$$

Onde SD_k é o saldo devedor após o pagamento da k -ésima prestação.

```
def tabela_price(pv, i, n):  
    """  
    Gera a tabela completa de amortização pelo Sistema Price.  
    Retorna lista de dicionários com período, prestação, juros, amortização, saldo.  
    """  
    pmt = pv * (i * (1 + i) ** n) / ((1 + i) ** n - 1)  
    sd = pv  
    tabela = []  
    total_juros = 0  
    for periodo in range(1, n + 1):  
        juros = sd * i  
        amortizacao = pmt - juros  
        sd -= amortizacao  
        total_juros += juros  
        tabela.append({  
            'periodo': periodo,  
            'prestacao': round(pmt, 2),  
            'juros': round(juros, 2),  
            'amortizacao': round(amortizacao, 2),  
            'saldo_devedor': round(abs(sd), 2) if sd < 0.01 else round(sd, 2)  
        })  
    return tabela, round(pmt * n - pv, 2)  
  
# Exemplo: R$50.000 em 12x a 2% a.m.
```

```

pv, i, n = 50000, 0.02, 12
tabela, total_juros = tabela_price(pv, i, n)

print(f"PRICE – R\${pv:,.2f} em {n}x a {i*100:.1f}% a.m.")
print(f"Prestação: R\${tabela[0]['prestacao']:.2f}")
print(f"Total de juros: R\${total_juros:,.2f}")
print()
print(f"{'Per':<5} {'Prestação':<12} {'Juros':<12} {'Amort':<12} {'Saldo Dev':<12}")
print("-" * 53)
for row in tabela:
    print(f"{'row['periodo']':<5} R\${row['prestacao']:<8.2f} R\${row['juros']:<8.2f} "
          f"R\${row['amortizacao']:<8.2f} R\${row['saldo_devedor']:<8.2f}")

```

5.3. SAC — Sistema de Amortizações Constantes

Características: - Amortização **constante** em todos os períodos - Prestações **decrescentes** (juros diminuem com o saldo) - Menor custo total (juros totais) que o Price para mesma taxa - Usado no SFH (Sistema Financeiro de Habitação) e financiamentos imobiliários

Cálculo da amortização:

$$A = \frac{PV}{n}$$

Evolução da dívida:

$$J_k = SD_{k-1} \times i$$

$$PMT_k = A + J_k$$

$$SD_k = SD_{k-1} - A$$

```

def tabela_sac(pv, i, n):
    """
    Gera tabela completa de amortização pelo SAC.
    """
    amortizacao = pv / n
    sd = pv
    tabela = []
    total_juros = 0
    for periodo in range(1, n + 1):
        juros = sd * i
        pmt = amortizacao + juros
        sd -= amortizacao
        total_juros += juros
        tabela.append({
            'periodo': periodo,
            'prestacao': round(pmt, 2),
            'juros': round(juros, 2),
            'amortizacao': round(amortizacao, 2),
            'saldo_devedor': round(sd, 2)
        })
    return tabela, round(total_juros, 2)

# Mesmo exemplo: R$50.000 em 12x a 2% a.m.
tabela_s, tj_sac = tabela_sac(pv, i, n)

print(f"SAC – R\${pv:,.2f} em {n}x a {i*100:.1f}% a.m.")

```



```

print(f"Amortização constante: R\${pv/n:.2f}")
print(f"1ª prestação: R\${tabela_s[0]['prestacao']:.2f}")
print(f"Última prestação: R\${tabela_s[-1]['prestacao']:.2f}")
print(f"Total de juros: R\${tj_sac:,.2f}")
print()
print(f"{'Per':<5} {'Prestação':<12} {'Juros':<12} {'Amort':<12} {'Saldo Dev':<12}")
print("-" * 53)
for row in tabela_s:
    print(f"{'row['periodo']':<5} R\${row['prestacao']:<8.2f} R\${row['juros']:<8.2f} "
          f"R\${row['amortizacao']:<8.2f} R\${row['saldo_devedor']:<8.2f}")

```

5.4. Comparação Price vs SAC

```

def comparar_sistemas(pv, i, n):
    """
    Comparação completa entre Price e SAC.
    """
    t_price, j_price = tabela_price(pv, i, n)
    t_sac, j_sac = tabela_sac(pv, i, n)

    print(f"{'Indicador':<35} {'PRICE':<15} {'SAC':<15}")
    print("-" * 65)
    print(f"{'Valor financiado':<35} R\${pv:<11,.2f} R\${pv:<,.2f}")
    print(f"{'Prestação inicial':<35} R\${t_price[0]['prestacao']:<10.2f} R\
    ${t_sac[0]['prestacao']:<.2f}")
    print(f"{'Prestação final':<35} R\${t_price[-1]['prestacao']:<10.2f} R\
    ${t_sac[-1]['prestacao']:<.2f}")
    print(f"{'Total prestações':<35} R\${pv + j_price:<10,.2f} R\${pv +
    j_sac:<,.2f}")
    print(f"{'Juros totais':<35} R\${j_price:<10,.2f} R\${j_sac:<,.2f}")
    print(f"{'Diferença de juros':<35} {'---':<15} R\${j_price - j_sac:<,.2f}")

    # Gráfico comparativo
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(range(1, n+1), [r['prestacao'] for r in t_price], 'b-o', label='Price')
    plt.plot(range(1, n+1), [r['prestacao'] for r in t_sac], 'r-s', label='SAC')
    plt.xlabel('Período'); plt.ylabel('Prestação (R$)')
    plt.title('Evolução das Prestações'); plt.legend(); plt.grid(alpha=0.3)

    plt.subplot(1, 2, 2)
    plt.plot(range(1, n+1), [r['saldo_devedor'] for r in t_price], 'b--',
    label='Price')
    plt.plot(range(1, n+1), [r['saldo_devedor'] for r in t_sac], 'r-', label='SAC')
    plt.xlabel('Período'); plt.ylabel('Saldo Devedor (R$)')
    plt.title('Evolução do Saldo Devedor'); plt.legend(); plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()

```

```
comparar_sistemas(50000, 0.02, 12)
```

5.5. SAM — Sistema de Amortização Mista

O SAM é a **média aritmética** entre Price e SAC:

$$PMT_k^{SAM} = \frac{PMT_k^{Price} + PMT_k^{SAC}}{2}$$

$$A_k^{SAM} = \frac{A_k^{Price} + A_k^{SAC}}{2}$$

5.6. Sistema Americano

Características: - Pagamento **apenas dos juros** durante o período - Principal (amortização total) pago ao **final** do contrato - Usado em debêntures, títulos de dívida (bond), operações interbancárias

$$J_k = PV \times i \quad (\text{constante para todos os períodos})$$

$$A_n = PV \quad (\text{apenas na última prestação})$$

5.7. Custo Efetivo Total (CET)

O CET vai além da taxa de juros, incluindo todos os encargos, tributos, tarifas e seguros:

$$CET = i_{efetiva} \quad \text{que iguala } PV_{recebido} \text{ a todos os pagamentos futuros}$$

```
def calcular_cet(pv_recebido, pagamentos, periodos):
    """
    Calcula o CET (Custo Efetivo Total) usando método iterativo.
    pagamentos: lista de valores pagos (positivos)
    periodos: lista de períodos correspondentes
    """

    def cet_funcao(taxa):
        vp_pagamentos = sum(pmt / (1 + taxa) ** t for pmt, t in zip(pagamentos,
periodos))
        return vp_pagamentos - pv_recebido

    # Método da bissecção para encontrar a taxa
    a, b = 0.0001, 1.0
    for _ in range(100):
        m = (a + b) / 2
        if cet_funcao(m) * cet_funcao(a) > 0:
            a = m
        else:
            b = m
    return (a + b) / 2

# Empréstimo de R$10.000 com 12 prestações de R$950 (já incluindo tarifas)
pv = 10000
pmt = 950
n = 12

# Simular CET
taxa_cet = calcular_cet(pv, [pmt] * n, list(range(1, n+1)))
print(f"CET mensal: {taxa_cet*100:.4f}%")
print(f"CET anual: {(1 + taxa_cet)**12 - 1}*100:.2f}%")

# Comparar com taxa de juros pura
i_puro = (i * (1 + i) ** n) / ((1 + i) ** n - 1) # taxa que geraria PMT de R$950
taxa_pura = ((pmt * ((1 + 0.02) ** n - 1) / (0.02 * (1 + 0.02) ** n)) / pv) ** (1/n)
- 1
```

Capítulo 6 — Inflação e Correção Monetária

6.1. Conceito de Inflação

Inflação é o aumento persistente e generalizado dos preços em uma economia. Seu efeito imediato é a **redução do poder de compra** da moeda.

Índices de preços no Brasil: - IPCA (IBGE) — índice oficial, usado para meta de inflação - INPC (IBGE) — foco em famílias de baixa renda - IGP-M (FGV) — usado em aluguéis e contratos - INCC (FGV) — custo da construção civil

6.2. Taxa Nominal vs Taxa Real (Equação de Fisher)

A relação entre taxa nominal (i_n), taxa real (i_r) e inflação (π):

$$(1 + i_n) = (1 + i_r) \times (1 + \pi)$$

Derivando:

$$i_r = \frac{1 + i_n}{1 + \pi} - 1$$

$$i_n = i_r + \pi + i_r \times \pi$$

Aproximação linear (válida para taxas baixas): $i_n \approx i_r + \pi$

```
def fisher(i_nominal=None, i_real=None, inflacao=None):
    """
    Calcula qualquer termo faltante da equação de Fisher.
    (1 + i_n) = (1 + i_r) * (1 + pi)
    """
    if i_nominal is None:
        return (1 + i_real) * (1 + inflacao) - 1
    elif i_real is None:
        return (1 + i_nominal) / (1 + inflacao) - 1
    elif inflacao is None:
        return (1 + i_nominal) / (1 + i_real) - 1

# Exemplo: CDI rendeu 13,25% com IPCA de 4,50%
i_nom, pi = 0.1325, 0.045
i_real = fisher(i_nominal=i_nom, inflacao=pi)
aprox = i_nom - pi
print(f"Rentabilidade nominal: {i_nom*100:.2f}%")
print(f"Inflação (IPCA): {pi*100:.2f}%")
print(f"Rentabilidade real: {i_real*100:.2f}%")
print(f"Aproximação ({i_nom*100:.0f}% - {pi*100:.0f}%): {aprox*100:.2f}%")
print(f"Erro da aproximação: {(i_real - aprox)*100:.2f} p.p.")

Rentabilidade nominal: 13.25%
Inflação (IPCA): 4.50%
Rentabilidade real: 8.37%
Aproximação (13% - 5%): 8.75%
Erro da aproximação: -0.38 p.p.
```

6.3. Correção Monetária

Valor corrigido por um índice de preços:

$$V_{\text{corrigido}} = V_{\text{original}} \times \prod_{t=1}^n (1 + \pi_t)$$

```
def corrigir_por_indice(valor_original, indices):
    """
    Corrige valor por uma série de índices de inflação.
    indices: lista de variações percentuais decimais
    """
    fator_acumulado = 1
    for idx, inf in enumerate(indices):
        fator_acumulado *= (1 + inf)
        print(f"  Mês {idx+1:2d}: inflação {inf*100:.2f}%, fator acumulado = {fator_acumulado:.4f}")
    return valor_original * fator_acumulado

# Aluguel de R$2.000 corrigido por 6 meses de IPCA
valor = 2000
inflacoes_mensais = [0.0056, 0.0048, 0.0062, 0.0039, 0.0051, 0.0043]

print(f"Valor original: R${valor:,.2f}")
print("Correção:")
valor_corrigido = corrigir_por_indice(valor, inflacoes_mensais)
print(f"\nValor corrigido (6 meses): R${valor_corrigido:,.2f}")
```

6.4. Taxa Pré-fixada vs Pós-fixada

Título Pré-fixado: a taxa é conhecida no momento da aplicação:

$$FV = PV \times (1 + i)^n$$

Título Pós-fixado: atrelado a um indexador + spread:

$$FV = PV \times (1 + indexador + spread)^n$$

Mas o indexador é desconhecido a priori, então usa-se projeções.

```
def comparar_prefixado_posfixado(pv, i_prefix, cdi_projetado, spread, n_dias):
    """
    Compara retorno de título pré e pós-fixado.
    """
    # Pré-fixado
    vf_pre = pv * (1 + i_prefix) ** (n_dias / 252)

    # Pós-fixado (CDI + spread)
    cdi_diario = (1 + cdi_projetado) ** (1/252) - 1
    taxa_pos_diaria = (1 + cdi_diario) * (1 + spread) - 1
    vf_pos = pv * (1 + taxa_pos_diaria) ** n_dias

    print(f"Pré-fixado ({i_prefix*100:.1f}% a.a.): R${vf_pre:,.2f}")
    print(f"Pós-fixado (CDI {cdi_projetado*100:.1f}% + spread {spread*100:.1f}%): R\
${vf_pos:,.2f}")
    print(f"Diferença: R${vf_pos - vf_pre:,.2f}")

    if vf_pre > vf_pos:
        print("Pré-fixado vale mais a pena (cenário de queda de juros)")
    else:
        print("Pós-fixado vale mais a pena (cenário de alta de juros)")

comparar_prefixado_posfixado(10000, 0.12, 0.1325, 0.005, 252)
```

Capítulo 7 – Análise de Investimentos

7.1. Valor Presente Líquido (VPL / NPV)

O VPL é a soma de todos os fluxos de caixa descontados a valor presente pela taxa mínima de atratividade (TMA):

$$VPL = \sum_{t=0}^n \frac{FC_t}{(1 + TMA)^t}$$

Critério de decisão: - $VPL > 0 \rightarrow$ investimento **viável** (cria valor) - $VPL = 0 \rightarrow$ indiferente - $VPL < 0 \rightarrow$ **inviável** (destrói valor)

```
def vpl(taxa, fluxos):
    """
    Calcula o Valor Presente Líquido.
    fluxos[0] é o investimento inicial (negativo).
    """
    return sum(fl / (1 + taxa) ** t for t, fl in enumerate(fluxos))

# Projeto A: investir R$15.000, receber R$4.500/ano por 5 anos, TMA=10%
fluxos_a = [-15000, 4500, 4500, 4500, 4500, 4500]
tma = 0.10
vpl_a = vpl(tma, fluxos_a)

# Projeto B: investir R$12.000, receber R$3.800/ano por 5 anos
fluxos_b = [-12000, 3800, 3800, 3800, 3800, 3800]
vpl_b = vpl(tma, fluxos_b)

print(f"Projeto A: VPL = R${vpl_a:.2f} {'VIÁVEL' if vpl_a > 0 else 'INVIÁVEL'}")
print(f"Projeto B: VPL = R${vpl_b:.2f} {'VIÁVEL' if vpl_b > 0 else 'INVIÁVEL'}")
print(f"Melhor projeto: {'A' if vpl_a > vpl_b else 'B'}")

Projeto A: VPL = R$2,058.58 VIÁVEL
Projeto B: VPL = R$2,404.35 VIÁVEL
Melhor projeto: B
```

7.2. Taxa Interna de Retorno (TIR / IRR)

A TIR é a taxa de desconto que **zera o VPL**:

$$0 = \sum_{t=0}^n \frac{FC_t}{(1 + TIR)^t}$$

Critério de decisão: $TIR > TMA \rightarrow$ viável

```
def tir(fluxos, precisao=1e-10, max_iter=10000):
    """
    Calcula a TIR pelo método de Newton-Raphson.
    """
    taxa = 0.1 # chute inicial
    for _ in range(max_iter):
        vp = sum(fl / (1 + taxa) ** t for t, fl in enumerate(fluxos))
        derivada = sum(-t * fl / (1 + taxa) ** (t + 1) for t, fl in
            enumerate(fluxos))
        if abs(derivada) < precisao:
            break
        taxa_nova = taxa - vp / derivada
        if abs(taxa_nova - taxa) < precisao:
```

```

        return taxa_nova
    taxa = taxa_nova
    return taxa

tir_a = tir(fluxos_a)
tir_b = tir(fluxos_b)

print(f"Projeto A: TIR = {tir_a*100:.2f}% a.a. (TMA={tma*100:.0f}%)")
print(f"Projeto B: TIR = {tir_b*100:.2f}% a.a. (TMA={tma*100:.0f}%)")
print(f"Decisão A: {'VIÁVEL' if tir_a > tma else 'INVIÁVEL'}")
print(f"Decisão B: {'VIÁVEL' if tir_b > tma else 'INVIÁVEL'}")

```

7.3. Limitações da TIR

1. **Múltiplas TIRs:** quando o fluxo de caixa muda de sinal mais de uma vez, podem existir múltiplas raízes
2. **TIR vs VPL:** a TIR pode classificar projetos incorretamente quando há diferença de escala ou prazo
3. **Taxa de reinvestimento:** a TIR assume que os fluxos intermediários são reinvestidos à própria TIR

```

# Exemplo de múltiplas TIRs: fluxo não convencional
fluxos_ao_convencionais = [-1000, 2500, -1500]
# Este fluxo pode ter 2 TIRs diferentes

# TIR Modificada (MTIR): assume reinvestimento à TMA
def mtir(fluxos, taxa_reinvestimento, taxa_financiamento=None):
    if taxa_financiamento is None:
        taxa_financiamento = taxa_reinvestimento
    vp_negativos = sum(min(fl, 0) / (1 + taxa_financiamento) ** t for t, fl in
enumerate(fluxos))
    vf_positivos = sum(max(fl, 0) * (1 + taxa_reinvestimento) ** (len(fluxos) - 1 -
t)
                        for t, fl in enumerate(fluxos))
    return (vf_positivos / (-vp_negativos)) ** (1 / (len(fluxos) - 1)) - 1

mtir_a = mtir(fluxos_a, tma)
print(f"MTIR do Projeto A: {mtir_a*100:.2f}% a.a.")

```

7.4. Payback Simples e Descontado

Payback Simples: tempo necessário para recuperar o investimento **sem** considerar o custo do capital.

$$Payback = \text{menor } n \text{ tal que } \sum_{t=0}^n FC_t \geq 0$$

Payback Descontado: considera o valor do dinheiro no tempo.

$$Payback_D = \text{menor } n \text{ tal que } \sum_{t=0}^n \frac{FC_t}{(1 + TMA)^t} \geq 0$$

```

def payback(fluxos, taxa=None):
    """
    Calcula payback simples ou descontado.
    """
    acum = 0
    for t, fl in enumerate(fluxos):

```

```

    if taxa is None:
        fl_descontado = fl
    else:
        fl_descontado = fl / (1 + taxa) ** t
    acum += fl_descontado
    if acum >= 0 and t > 0:
        # Interpolação linear para fração do período
        if t > 0:
            acum_anterior = acum - fl_descontado
            fracao = -acum_anterior / (fl_descontado - acum_anterior) if
fl_descontado != 0 else 1
            return t - 1 + fracao
        return t
    return None

print(f"Payback simples (Proj A): {payback(fluxos_a):.2f} anos")
print(f"Payback descontado (Proj A): {payback(fluxos_a, 0.10):.2f} anos")

```

7.5. Índice de Lucratividade (IL)

Mede a relação benefício-custo:

$$IL = \frac{\sum_{t=1}^n \frac{FC_t}{(1+TMA)^t}}{|FC_0|}$$

Critério: $IL > 1 \rightarrow$ projeto viável

```

def indice_lucratividade(taxa, fluxos):
    vp_entradas = sum(fl / (1 + taxa) ** t for t, fl in enumerate(fluxos) if t > 0)
    investimento = abs(min(0, fluxos[0]))
    return vp_entradas / investimento

il_a = indice_lucratividade(0.10, fluxos_a)
il_b = indice_lucratividade(0.10, fluxos_b)
print(f"IL A: {il_a:.3f} {'VIÁVEL' if il_a > 1 else 'INVIÁVEL'}")
print(f"IL B: {il_b:.3f} {'VIÁVEL' if il_b > 1 else 'INVIÁVEL'}")

```

7.6. VPLA — Valor Presente Líquido Anualizado

Converte o VPL em uma série uniforme equivalente ao longo da vida do projeto:

$$VPLA = VPL \times \frac{i \times (1 + i)^n}{(1 + i)^n - 1}$$

Útil para comparar projetos com **diferentes prazos**.

```

def vpla(taxa, fluxos, n):
    vpl_ = vpl(taxa, fluxos)
    fator = (taxa * (1 + taxa) ** n) / ((1 + taxa) ** n - 1)
    return vpl_ * fator

# Projetos com prazos diferentes
# Proj X: 3 anos, VPL = R$10.000
# Proj Y: 5 anos, VPL = R$14.000
taxa = 0.10
vpla_x = vpla(taxa, [0, 0, 0, 10000], 3) # simplificado
vpla_y = vpla(taxa, [0, 0, 0, 0, 0, 14000], 5)

# Na verdade VPL já calculado:

```

```

vpl_x, n_x = 10000, 3
vpl_y, n_y = 14000, 5

vpla_x = vpl_x * (taxa * (1+taxa)**n_x) / ((1+taxa)**n_x - 1)
vpla_y = vpl_y * (taxa * (1+taxa)**n_y) / ((1+taxa)**n_y - 1)

print(f"Projeto X (3 anos): VPL = R\${vpl_x:,.2f}, VPLA = R\${vpla_x:,.2f}/ano")
print(f"Projeto Y (5 anos): VPL = R\${vpl_y:,.2f}, VPLA = R\${vpla_y:,.2f}/ano")
print(f"Melhor pelo VPLA: {'X' if vpla_x > vpla_y else 'Y'}")

```

7.7. Análise de Sensibilidade

Testa como o VPL varia em função de mudanças nas variáveis-chave (receita, custo, taxa).

```

def analise_sensibilidade(fluxos_base, tma_base, variacao=0.3):
    """
    Analisa sensibilidade do VPL a variações nas principais variáveis.
    """
    importancias = {}
    variaveis = ['investimento', 'receita_anual', 'tma']

    print(f"{'Variável':<20} {'-30%':<12} {'-15%':<12} {'Base':<12} {'+15%':<12} {'+30%':<12}")
    print("-" * 68)
    for var in variaveis:
        valores = []
        for delta in [-0.3, -0.15, 0, 0.15, 0.3]:
            fluxos_mod = fluxos_base.copy()
            tma_mod = tma_base
            if var == 'investimento':
                fluxos_mod[0] = fluxos_base[0] * (1 + delta)
            elif var == 'receita_anual':
                for t in range(1, len(fluxos_mod)):
                    fluxos_mod[t] = fluxos_base[t] * (1 + delta)
            elif var == 'tma':
                tma_mod = tma_base + delta
            vpl_ = vpl(tma_mod, fluxos_mod)
            valores.append(vpl_)
        print(f"{var:<20} " + " ".join(f"R\${v:<8,.0f}" for v in valores))

analise_sensibilidade([-15000, 4500, 4500, 4500, 4500, 4500], 0.10)

```

Capítulo 8 – Tabela Resumo de Fórmulas

Conceito	Fórmula	Observação
Juros Simples	$J = PV \times i \times n$	Crescimento linear
Montante (Simples)	$FV = PV \times (1 + i \times n)$	PA
Juros Compostos	$J = PV \times [(1 + i)^n - 1]$	Crescimento exponencial
Montante (Composto)	$FV = PV \times (1 + i)^n$	PG
Valor Presente	$PV = FV / (1 + i)^n$	Descapitalização
Taxa Equivalente (Comp)	$i_{maior} = (1 + i_{menor})^k - 1$	Não confundir com proporcional
Nominal → Efetiva	$i_{ef} = (1 + i_{nom}/k)^k - 1$	k = capitalizações
Série Postecipada	$PMT = PV \times \frac{i(1+i)^n}{(1+i)^n - 1}$	Sem entrada
Série Antecipada	$PV_{ant} = PV_{post} \times (1 + i)$	Com entrada
Perpetuidade	$PV = PMT / i$	Pagamentos infinitos
Gordon	$PV = PMT / (i - g)$	Com crescimento
Price	Prestações fixas	Juros decrescentes, amort. crescente
SAC	Amortização fixa	Prestações decrescentes
VPL	$\sum FC_t / (1 + TMA)^t$	> 0 = viável
TIR	$\sum FC_t / (1 + TIR)^t = 0$	$> TMA$ = viável
Payback	Tempo de recuperação	Sem considerar custo de capital
Fisher	$(1 + i_n) = (1 + i_r)(1 + \pi)$	Relação nominal-real
IL	$VP_{entradas} / \text{investimento} $	> 1 = viável

Capítulo 9 – Exercícios Resolvidos

E1. Um capital de R\$ 8.000 foi aplicado a juros simples de 1,8% ao mês durante 15 meses. Calcule o montante e os juros.

$$FV = 8000 \times (1 + 0,018 \times 15) = 8000 \times 1,27 = 10.160$$

$$J = 10.160 - 8.000 = 2.160$$

```
pv, i, n = 8000, 0.018, 15
fv = pv * (1 + i * n)
print(f"Montante: R${fv:.2f}, Juros: R${fv - pv:.2f}")
```

E2. Qual o capital que, aplicado a 2,5% ao mês por 8 meses, rende juros compostos de R\$ 2.500?

$$PV = \frac{J}{(1 + i)^n - 1} = \frac{2500}{(1,025)^8 - 1}$$

```
j, i, n = 2500, 0.025, 8
pv = j / ((1 + i) ** n - 1)
print(f"Capital necessário: R${pv:.2f}")
```

E3. Uma loja vende um produto por R\$ 1.200 à vista ou em 3 parcelas iguais sem entrada, a juros de 3% ao mês. Qual o valor de cada parcela?

$$PMT = 1200 \times \frac{0,03 \times (1,03)^3}{(1,03)^3 - 1}$$

```
pv, i, n = 1200, 0.03, 3
pmt = pv * (i * (1 + i) ** n) / ((1 + i) ** n - 1)
print(f"Valor de cada parcela: R\${pmt:.2f}")
print(f"Total a prazo: R\${pmt * n:.2f}")
print(f"Juros pagos: R\${pmt * n - pv:.2f}")
```

E4. Um título de R\$ 10.000 com vencimento em 60 dias é descontado a uma taxa de desconto comercial de 3,5% ao mês. Qual o valor líquido?

$$PV = 10000 \times (1 - 0,035 \times 2)$$

```
fv, i, n = 10000, 0.035, 2
pv = fv * (1 - i * n)
print(f"Valor líquido: R\${pv:.2f}")
```

E5. Um investimento de R\$ 25.000 promete retornos de R\$ 7.000/ano por 5 anos. TMA = 12% a.a. Calcule VPL, TIR e Payback.

```
fluxos = [-25000, 7000, 7000, 7000, 7000, 7000]
tma = 0.12
```

```
vpl_ = vpl(tma, fluxos)
tir_ = tir(fluxos)
pb = payback(fluxos)
pb_desc = payback(fluxos, tma)
```

```
print(f"VPL: R\${vpl_:.2f}")
print(f"TIR: {tir_*100:.2f}% a.a.")
print(f"Payback simples: {pb:.2f} anos")
print(f"Payback descontado: {pb_desc:.2f} anos")
print(f"Decisão: {'VIÁVEL' if vpl_ > 0 and tir_ > tma else 'INVIÁVEL'})
```

E6. Uma empresa tem WACC de 14%, capital próprio de R\$ 80 milhões e dívida de R\$ 40 milhões. O custo do capital próprio é 16% e o custo da dívida bruto é 10% (IR = 34%). Verifique se o WACC está correto.

$$WACC = \frac{80}{120} \times 0,16 + \frac{40}{120} \times 0,10 \times (1 - 0,34)$$

```
pl, divida = 80, 40
ke, kd, ir = 0.16, 0.10, 0.34
v = pl + divida
wacc_ = (pl/v) * ke + (divida/v) * kd * (1 - ir)
print(f"WACC: {wacc_*100:.2f}%")
```

E7. Um título pós-fixado rende 110% do CDI. Se o CDI está em 13,25% a.a. e o investimento é de R\$ 5.000 por 252 dias, qual o valor de resgate?

```

pv, cdi, percentual = 5000, 0.1325, 1.10
taxa_aplicada = cdi * percentual
vf = pv * (1 + taxa_aplicada)
print(f"Taxa aplicada: {taxa_aplicada*100:.2f}% a.a.")
print(f"Valor de resgate: R${vf:.2f}")

```

Capítulo 10 – Glossário

Termo	Definição
Amortização	Parcela da prestação que abate o saldo devedor
Capitalização	Processo de incorporação dos juros ao capital
CET	Custo Efetivo Total – inclui todos os encargos da operação
Desconto	Abatimento pelo pagamento antecipado
Equivalência de taxas	Duas taxas produzem o mesmo montante no mesmo prazo
Fluxo de caixa	Representação das entradas e saídas de dinheiro no tempo
FRC	Fator de Recuperação de Capital – usado para calcular prestações
Juros	Remuneração do capital emprestado
Montante	Capital + Juros acumulados
Payback	Tempo de recuperação do investimento
Perpetuidade	Série infinita de pagamentos
Prazo médio	Média ponderada dos prazos de vários títulos
Price	Sistema de amortização com prestações fixas
SAC	Sistema de Amortização Constante
TIR	Taxa Interna de Retorno
TMA	Taxa Mínima de Atratividade
VPL	Valor Presente Líquido
WACC	Custo Médio Ponderado de Capital